

UNIVERSITY OF GHANA

Department of Computer Science

CSCD602: ADVANCED SOFTWARE ENGINEERING

Individual Project-Based Examination

PROJECT DOCUMENTATION

Part 1: Requirements, Effort Estimation, System Design & Technical Debt Plan

Project Title: WasteTrack Ghana

A Web-Based Waste Collection Request & Tracking System

Student Name: [ATTOR JOHN]

Student ID: [22424714]

Examiner: Prof. Solomon Mensah

Academic Year: 2025/2026, Second Semester

Document Purpose & Scope of This Deliverable

This document constitutes the pre-implementation deliverable for the WasteTrack Ghana capstone examination. It consolidates and formalises the requirements engineering, software effort estimation, system analysis and design, and technical debt planning that were carried out before any code was written, in direct response to the examination instruction that requirements and effort estimation must guide implementation decisions.

It is deliberately produced before implementation begins. Its purpose is to fix the scope, architecture and data model that implementation will follow, and to provide the documented evidence of process that the examiner will assess under the Requirements Engineering & SRS, Software Effort Estimation, System Analysis & Design, and Technical Debt Identification & Management components of the marking scheme.

This document will later be merged into the final consolidated Project Documentation submitted at the end of the examination window, alongside the Testing Report, Technical Debt Plan (finalised), User Manual, and Deployment & Source Links file.

Chapter 1: Requirements Engineering

1.1 Problem Statement

In many Ghanaian residential communities, household waste collection is coordinated informally: residents call, text or physically flag down collection staff, and requests are tracked on paper or not tracked at all. This produces missed collections, no visibility into request status for residents, and no consolidated view for waste-management staff of outstanding, assigned and completed jobs.

WasteTrack Ghana addresses this gap with a lightweight, web-based system that lets a resident submit a waste-collection request and track its status online, and lets an administrator view all outstanding requests, assign a collector, and update status as the job progresses through its lifecycle.

1.2 Aim and Objectives

Aim: To design and deploy a functional web application that digitises the request-and-track workflow for household waste collection, within the constraints of a 48-hour individual examination.

Objectives:

- Allow a resident to register, log in, and submit a waste-collection request with the details a collector needs.
- Allow a resident to view the status of their own requests without needing to phone or visit an office.
- Allow an administrator to see all outstanding requests, assign a collector, and progress a request through a defined status lifecycle.
- Enforce role-based access so residents and administrators only see and do what their role permits.
- Deploy the application so that it is reachable over the internet by the examiner, with working test credentials.

1.3 Stakeholders

Stakeholder	Role / Interest
Resident (end user)	Submits collection requests and needs visibility into whether/when waste will be collected.
Administrator (waste management staff)	Coordinates collection: reviews incoming requests, assigns collectors, updates status.
Collector (field staff)	Physically carries out the collection. Represented as a data entity assigned to requests; does not log into the system in Version 1.
Examiner (Prof. Solomon Mensah)	Assesses the artefact against the CSCD602 marking scheme; needs a working, explainable, deployed system.
Developer (student)	Individually responsible for the full lifecycle: requirements through deployment, within 48 hours.

1.4 Functional Requirements

Functional requirements are grouped by capability area and each is given an ID used for traceability into design, implementation and testing.

ID	Requirement	Priority
FR-01	The system shall allow a new resident to register with name, email and password.	Must
FR-02	The system shall allow a registered user to log in and log out securely.	Must
FR-03	The system shall enforce role-based authorisation distinguishing Resident and Administrator accounts.	Must
FR-04	The system shall allow a resident to submit a collection request specifying address, waste type, preferred date and description.	Must
FR-05	The system shall validate collection-request input before it is saved (required fields, sensible date, non-empty address).	Must
FR-06	The system shall allow a resident to view a list of their own requests and each request's current status.	Must
FR-07	The system shall allow an administrator to view all collection requests submitted by all residents.	Must
FR-08	The system shall allow an administrator to assign a collector to a pending request.	Must
FR-09	The system shall allow an administrator to update a request's status as it progresses (Pending → Assigned → In Progress → Collected), or cancel it.	Must
FR-10	The system shall present a resident-facing dashboard summarising the resident's own request history.	Should
FR-11	The system shall allow filtering/searching of requests by status on the admin dashboard.	Should
FR-12	The system shall display simple aggregate statistics (e.g. counts by status) on the admin dashboard.	Should
FR-13	The system shall notify a resident (in-app) when their request status changes.	Could
FR-14	The system shall allow a resident to attach a photo to a collection request.	Could
FR-15	The system shall support GPS-based address capture instead of free-text address.	Won't (v1)

1.5 Non-Functional Requirements

ID	Requirement	Category
NFR-01	Passwords shall be stored as salted hashes, never in plain text.	Security
NFR-02	Only authenticated users may access dashboard, request or admin routes; unauthenticated requests shall be redirected to login.	Security
NFR-03	An administrator route shall not be reachable by a resident account (server-side role check, not just hidden UI).	Security
NFR-04	Core pages shall render usably on both desktop and mobile-width viewports.	Usability
NFR-05	A typical request-submission or dashboard page shall load in under ~2 seconds on a normal broadband connection.	Performance
NFR-06	The codebase shall be organised into models, routes and templates to keep it maintainable within the time constraint.	Maintainability
NFR-07	The system shall remain available and reachable at a public URL for the duration of grading.	Availability
NFR-08	All third-party libraries and frameworks used shall be acknowledged in the documentation.	Compliance

1.6 Requirements Prioritisation (MoSCoW)

Given the 48-hour constraint, requirements were prioritised using MoSCoW so that estimation (Chapter 2) could be used to make a defensible scope cut rather than an arbitrary one.

Category	Included Requirements
Must Have (Version 1 / MVP)	FR-01 to FR-09 registration, login, role authorisation, submit request, validation, view/track requests, admin view, assign collector, update status.
Should Have (implemented only if core is stable)	FR-10, FR-11, FR-12 resident dashboard summary, search/filter, admin statistics.
Could Have (not depended upon)	FR-13, FR-14 in-app notifications, photo upload.
Won't Have (explicitly out of Version 1)	Payment processing, GPS/live tracking, route optimisation, AI-based collector matching, native mobile application.

1.7 Scope Statement (Version 1)

The project will implement a web-based waste collection request and tracking system supporting two primary user roles: resident and administrator. The Version 1 (MVP) system supports authentication, waste-collection request submission, request tracking, collector assignment, status management, input validation and a responsive web interface. Advanced features payment, GPS/live tracking, route optimisation, automated collector matching, notifications and AI-based functionality are consciously excluded from Version 1 as a direct, documented consequence of the effort estimation in Chapter 2, not because they were overlooked.

Chapter 2 Software Effort Estimation

2.1 Estimation Technique Selected

Technique: Story Points, supported by Expert (analogy-based) Estimation to convert points into person-hours.

Justification: WasteTrack Ghana is a small, single-developer application built in a fixed, very short window. A parametric model such as COCOMO/COCOMO II depends on calibration against historical project data (e.g. organisational productivity factors) that does not exist for a first-time individual project of this kind, so it would produce a spuriously precise but poorly-grounded number. Function Point Analysis is more suited to larger, requirements-heavy systems and adds overhead disproportionate to a 48-hour build. Story points let requirements be compared for relative implementation difficulty, and expert estimation (the developer's own judgement, calibrated against similar past exercises) converts that relative sizing into an approximate hour figure. This combination is lightweight, fast to apply, and appropriate for the project's size and timeframe.

2.2 Work Breakdown & Story-Point Estimate

The Must-Have requirement set (Section 1.6) was broken into discrete work items and sized in story points, with an expert-estimated hour figure attached to each.

ID	Work Item	Story Pts	Hours
W01	Project setup & Git repository	1	1
W02	Database design & setup (SQLAlchemy models)	2	2
W03	User registration	2	2
W04	Login / logout	3	3
W05	Role-based authorisation	3	3
W06	Resident dashboard	3	3
W07	Create collection request	3	3
W08	View / track requests	3	3
W09	Admin dashboard	3	3
W10	Assign collector	2	2
W11	Update request status	2	2
W12	Input validation & error handling	2	2
W13	Responsive UI refinement	2	2
W14	Unit / integration testing	3	4
W15	Deployment configuration	3	3
W16	Production testing & fixes	2	2
	TOTAL	39	40

Table 2.1 Work breakdown for the Must-Have (Version 1) feature set: ≈ 40 story points / ≈ 40 implementation hours.

2.3 Reconciling the Estimate with the 48-Hour Budget

The raw implementation estimate of ~40 hours cannot, by itself, consume the full 48-hour examination window, because implementation is only one of the phases the examiner requires evidence of. The recommended examination schedule allocates time across all lifecycle phases:

Phase	Recommended Hours (Examiner's Guide)
Requirements & planning	6
Analysis & design	6
Implementation	20
Testing & refinement	6
Deployment	4
Documentation	6
TOTAL	48

The 40-hour raw implementation estimate is roughly double the 20 hours the recommended schedule allots to implementation. This mismatch is the trigger that forced the scope reduction described in Section 1.6 and 1.7 it is the direct, evidenced link between estimation and scope that the examination asks for.

2.4 Reconciled Project Effort Estimate

Phase	Estimated Effort
Requirements & SRS	4 hours
Effort estimation & scope definition	1 hour
Analysis & design	5 hours
Core implementation (Must-Have items only)	17 hours
Testing & refinement	6 hours
Deployment	3 hours
Documentation & finalisation	8 hours
TOTAL	44 hours

This leaves $48 - 44 = 4$ hours of contingency. A 48-hour project estimated at exactly 48 hours has zero room for the unexpected a stuck bug, a failed deployment, a re-run test suite so the reconciled estimate is deliberately kept under budget: ≈ 44 hours of planned work plus ≈ 4 hours of contingency, rather than 48 hours of planned work with none.

2.5 Estimated Development Duration

Total estimated duration: 44 hours of planned effort within the fixed 48-hour examination window (single developer, continuous individual effort, not elapsed calendar time).

2.6 Assumptions

- The application is a small prototype, not a production-scale commercial system.
- Established frameworks (Flask, SQLAlchemy, Bootstrap-level CSS) are used rather than building infrastructure from scratch.

- SQLite is sufficient for the prototype's data volume and concurrency needs.
- An existing PaaS deployment platform (Render) is used rather than self-managed infrastructure.
- Authentication is username/password based; no third-party identity provider is integrated.
- The system has exactly two roles: Resident and Administrator.
- The application excludes payment, GPS, SMS or other complex external integrations.
- Testing focuses on core workflows (auth, request submission, status transitions) rather than exhaustive edge-case coverage.
- Documentation is produced alongside development, not written from scratch after coding finishes.

Estimation is conditional, not absolute: given these assumptions and constraints, the current estimate is ≈ 44 hours. Changing an assumption (e.g. adding a third role, or requiring PostgreSQL from day one) would change the estimate.

2.7 Constraints

- Hard 48-hour examination limit the single overriding constraint.
- Individual work no collaboration on implementation is permitted.
- Limited development time available for any one feature.
- Limited time available for testing depth.
- Limited infrastructure a single free-tier deployment target.
- Limited acceptable system complexity, to keep the design explainable.
- Deployment must work and remain accessible for the duration of grading.
- Source code must be available to the examiner in a repository.
- The developer must understand every submitted line well enough to defend it in a viva voce.

The Viva Constraint

Unlike the time-based constraints above, this one is not measured in hours but shapes every estimation and scoping decision: the examiner may ask the developer to explain requirements, effort estimation, architecture, implementation decisions, testing and technical debt. Simplicity is therefore pursued not only to finish inside 48 hours, but to guarantee the developer can defend everything submitted.

2.8 How the Estimate Influenced Scope

This is the explicit requirements→estimation→scope link the marking scheme rewards. The sequence was:

- The Must-Have feature list (Section 1.6) was sized at ≈ 40 implementation hours (Section 2.2).
- That figure was checked against the 20 hours the recommended schedule allots to implementation, and found to be roughly double.
- Rather than compress every feature's quality to fit, features were removed from Version 1 using the MoSCoW classification already agreed in Section 1.6: Should-Have items (search/filter, dashboard statistics) become stretch goals attempted only if the core is stable; Could-Have items (notifications, photo upload) and Won't-Have items (payment, GPS, route optimisation, AI matching, native mobile app) are excluded from Version 1 outright.

- The reconciled 44-hour estimate (Section 2.4) reflects this reduced scope, not the original 40-hour Must-Have-only implementation figure plus untouched design/testing/deployment/documentation time.

This is a conscious exclusion, not an omission: the project is not saying "these features could not be built"; it is saying "these features were evaluated against the effort estimate and constraints, and deliberately excluded from Version 1." That is requirements and estimation actively driving implementation decisions.

Chapter 3: System Analysis & Design

Five design artefacts were selected as sufficient to communicate this system, in line with the examiner's guidance that not every possible diagram is required only those that best communicate the system: system architecture, use-case diagram, activity diagram, request-state diagram, and ER/database diagram, supported by UI wireframes and a deployment view.

3.1 System Architecture

WasteTrack Ghana follows a conventional three-layer web architecture: a presentation layer in the browser, an application layer in Flask, and a data layer in SQLite accessed through SQLAlchemy. The user never talks to the database directly — every request is mediated by the Flask application layer, which is where authentication, authorisation, validation and business logic live.

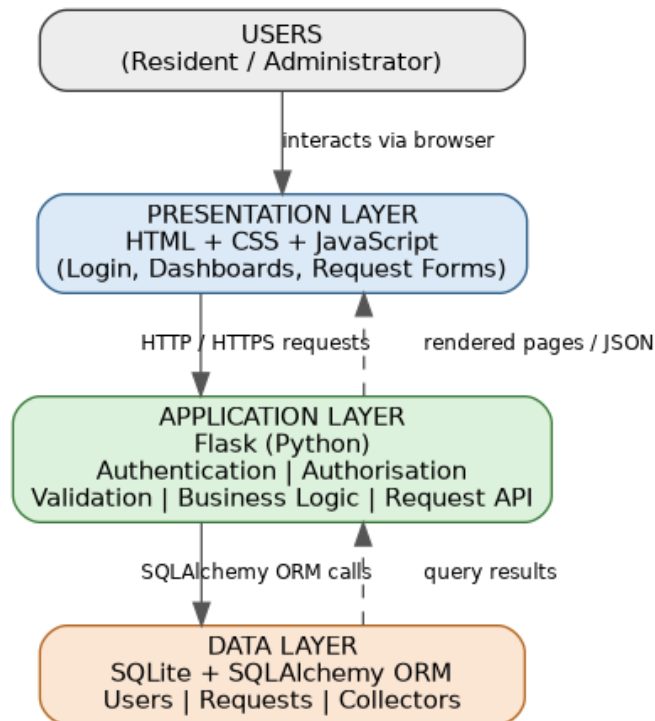


Figure 3.1 Three-layer system architecture.

3.2 Use-Case Diagram

Two actors interact with the system. The resident registers, logs in, submits and tracks their own requests. The administrator logs in, sees all requests, assigns collectors and updates status. This directly mirrors the Must-Have functional requirements FR-01 to FR-09.

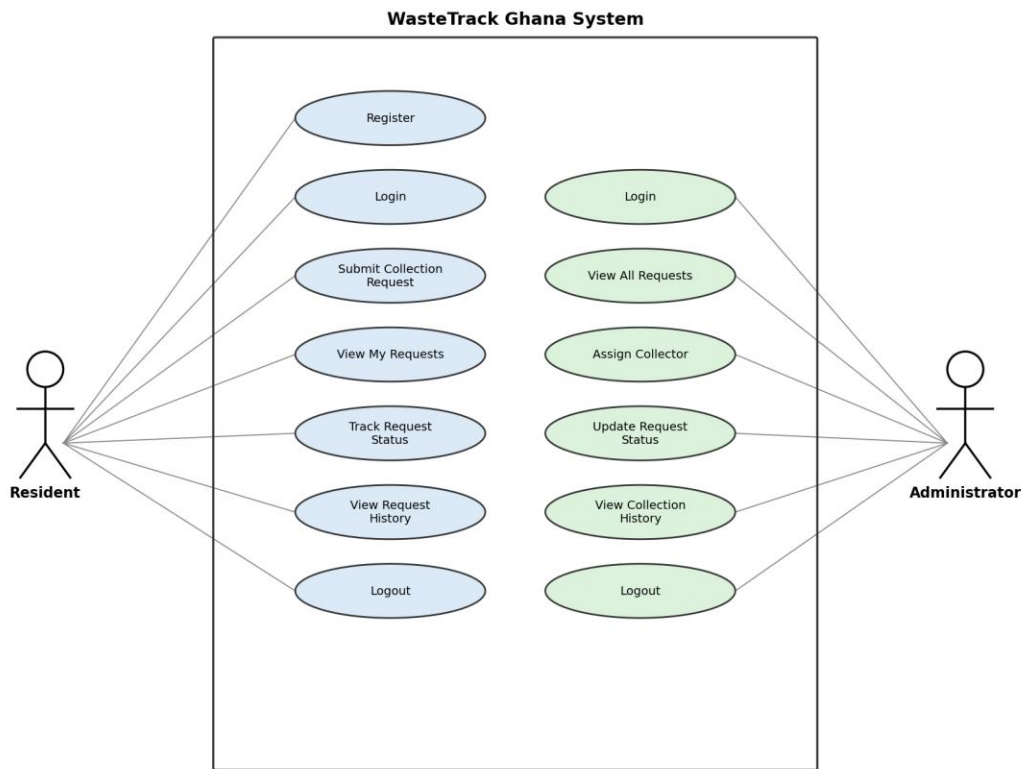


Figure 3.2 Use-case diagram: Resident and Administrator actors against the WasteTrack Ghana system boundary.

3.3 Activity Diagram Submit Collection Request

This is the system's central workflow. The key engineering decision embedded in this diagram is that validation happens before the request is persisted: an invalid submission returns the resident to the form with an error rather than writing bad data to the database (implements FR-05).

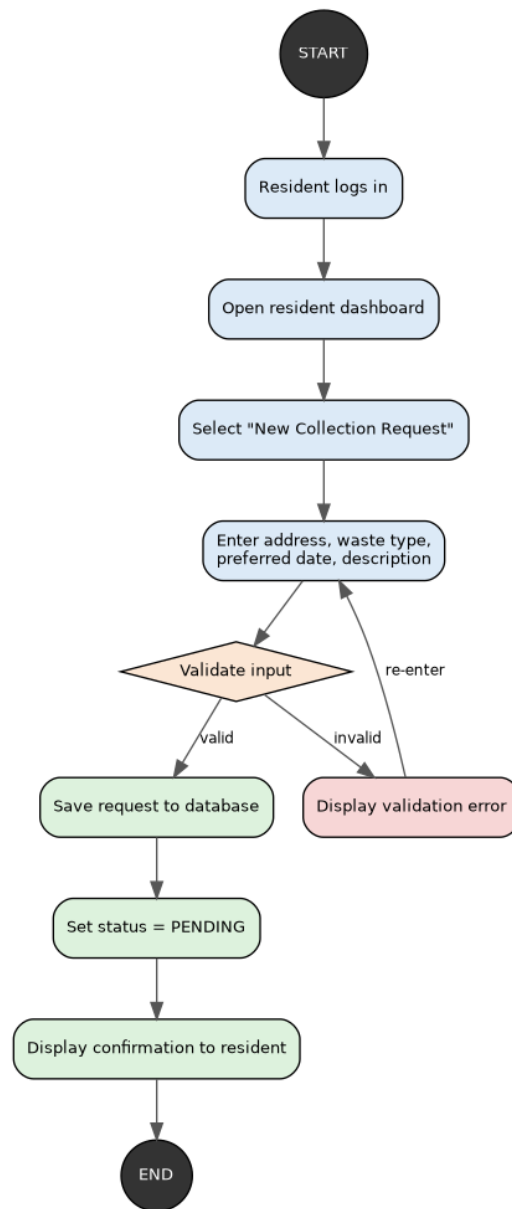


Figure 3.3 Activity diagram for submitting a collection request.

3.4 Request Lifecycle (State Diagram)

Every collection request carries a status field that moves through a small, explicit state machine. This state model is what implementation (FR-08, FR-09) and later testing are built against.

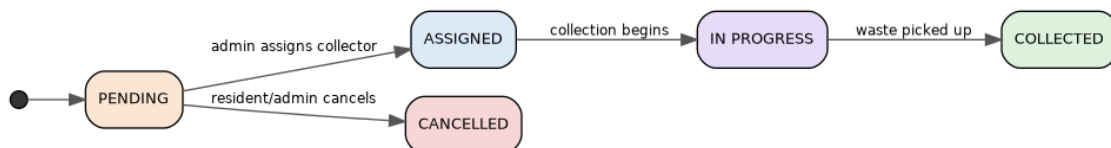


Figure 3.4 Request status lifecycle: *PENDING* → *ASSIGNED* → *IN PROGRESS* → *COLLECTED*, with *PENDING* → *CANCELLED* as an alternative path.

Trigger	From → To
Resident submits a new request	(start) → PENDING
Administrator assigns a collector	PENDING → ASSIGNED
Collection begins	ASSIGNED → IN PROGRESS
Waste is picked up	IN PROGRESS → COLLECTED
Resident or administrator cancels a pending request	PENDING → CANCELLED

3.5 ER / Database Diagram

The data model has three entities. A resident (User) can submit many Requests; a Collector can be assigned to many Requests. Deliberately absent from this schema: GPS coordinates, map location data, photo attachments and payment information because those capabilities were excluded from Version 1 in Section 1.6. The schema was intentionally limited to the information required by the prioritised Must-Have requirements; this is requirements directly shaping design.

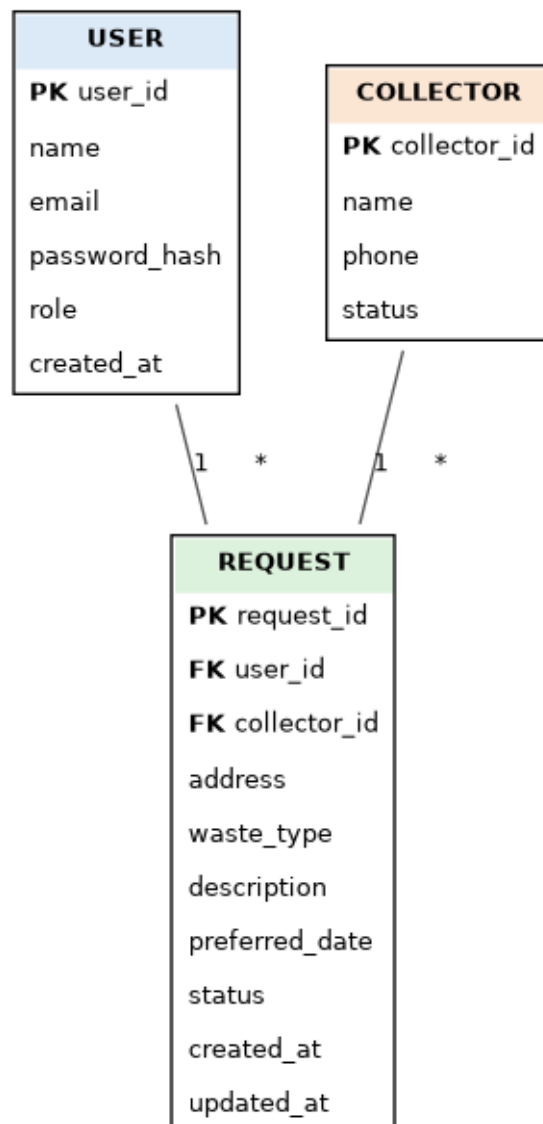


Figure 3.5 Entity-relationship diagram: USER (1) — (*) REQUEST (*) — (1) COLLECTOR.

3.6 UI Wireframes

The wireframes below are engineering wireframes, not visual design their purpose is to fix what fields and controls each page needs before implementation, not to finalise a visual style.

Login Page

WASTETRACK GHANA — Login	
Email:	
Password:	
[LOGIN]	
Don't have an account? Register	

Resident Dashboard

WasteTrack Ghana	John Logout
Welcome, John	
[+ New Collection Request]	
My Requests:	
#0012 Pending	
#0011 Assigned	
#0010 Collected	

New Collection Request

NEW COLLECTION REQUEST	
Address:	
Waste Type:	[Household Waste v]
Preferred Date:	
Description:	
[SUBMIT REQUEST]	

Admin Dashboard

ADMIN DASHBOARD				Logout
ID	Resident	Status	Collector	Action
----	-----	-----	-----	-----
001	John	Pending	—	Assign
002	Ama	Assigned	Kofi	Update
003	Kojo	Collected	Yaw	View

3.7 Deployment Architecture

In production, the Flask application runs behind a WSGI server (Gunicorn) on Render, with the SQLite file on Render's persistent disk. The examiner reaches the live application over the public internet exactly as a resident or administrator would.

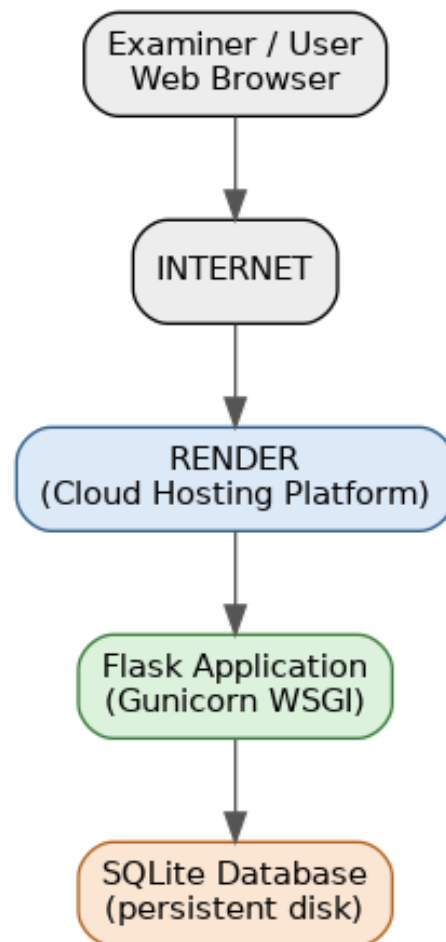


Figure 3.6 Deployment view on Render.

3.8 Technology Stack

Layer	Choice	Rationale
Development environment	VS Code	Free, lightweight, strong Python + Git tooling.
Language	Python	Prior familiarity from MSc/ML coursework; no new language to learn under time pressure.
Backend framework	Flask	Minimal, unopinionated; fast to stand up routes, auth and business logic.
ORM	SQLAlchemy	Avoids scattering raw SQL; models map cleanly to the ER diagram; eases a future move to PostgreSQL.
Database	SQLite	Zero-configuration, file-based, sufficient for prototype workload.

Layer	Choice	Rationale
Frontend	HTML + CSS + JavaScript (server-rendered)	The exam assesses engineering discipline, not frontend framework fluency; a simpler frontend frees time for testing, deployment and documentation.
Version control	Git	Commit history evidences incremental, disciplined development.
Remote repository	GitHub	Required so the examiner can access the source code.
Testing	pytest	Lightweight automated testing for core Flask routes and models.
Diagramming	diagrams.net / Graphviz	Used to produce the artefacts in this chapter.
Deployment	Render	Explicitly named as an acceptable platform by the examiner; supports Flask + persistent SQLite easily.

Chapter 4 Technical Debt Plan

The examination explicitly requires technical debt to be identified, prioritised and managed, with each significant item documented as Debt → Cause → Impact → Priority → Proposed Resolution. The debt below is deliberately accepted, not accidental: each item is a conscious trade-off made to fit the 48-hour constraint, tracked here so it can be repaid after submission.

4.1 Technical Debt Register

Debt	Cause	Impact	Priority	Proposed Resolution
SQLite as the database engine	48-hour constraint; small expected workload	Limited write concurrency and scalability	Medium	Migrate to PostgreSQL; SQLAlchemy models minimise rewrite effort
Server-rendered frontend (no SPA framework)	Faster to implement than a React/Vue frontend	Less interactive UX than a modern SPA	Low	Introduce React or Vue if the project grows
Manual collector assignment	Simpler to implement than an allocation algorithm	Administrator must assign every request manually	Medium	Add automatic assignment based on collector availability/location
Free-text address instead of GPS	Avoids map-API integration complexity	Location is not geographically precise for collectors	Medium	Integrate a mapping/geolocation API
No resident notifications	Time constraint	Residents must manually check status rather than being alerted	Low	Add email/SMS or in-app notifications on status change
Limited automated test coverage	48-hour constraint	Some regressions may go undetected	Medium	Expand pytest unit/integration coverage, especially around status transitions
Simple single-instance deployment	Small academic prototype, free-tier hosting	Not designed for high traffic or high availability	Medium	Introduce a scalable cloud architecture (managed DB, autoscaling) if usage grows
Basic session-based authentication	Sufficient for a prototype; avoids third-party auth-provider integration	Would need hardening (rate limiting, password policies, MFA) for production	Medium	Strengthen authentication before any real-world deployment

4.2 Debt Management Philosophy

The goal is not to eliminate all technical debt inside 48 hours that would be unrealistic and, if attempted, would itself be poor engineering judgement given the constraint. The goal is to demonstrate that debt is recognised, its cause and consequences are understood, it is prioritised rather than treated uniformly, and a credible plan exists to repay it.

Each item above is classified using the following distinction, as required by the examination:

Classification	Meaning	Applies to
Acceptable temporarily	Safe to ship as-is for the examination; no immediate risk to correctness or grading	Server-rendered frontend; no resident notifications
Scheduled for future resolution	Should be addressed in the next development cycle, not urgent for submission	SQLite → PostgreSQL; manual assignment; free-text address; limited test coverage; single-instance deployment
Critical / requires immediate attention	Would block correct grading or expose the system to obvious risk if unresolved	None identified the Must-Have scope (Section 1.6) was deliberately kept small enough that no debt item rises to this level

4.3 Technical Debt Repayment Plan (Post-Submission)

Priority	Item	Target Timeframe
High (do first)	Expand automated test coverage around status transitions and authorisation checks	Immediately after submission / first maintenance cycle
High	Migrate SQLite → PostgreSQL	First maintenance cycle
Medium	Automatic collector assignment (availability/location based)	Second iteration
Medium	Mapping/geolocation integration for addresses	Second iteration
Medium	Harden authentication (rate limiting, password policy)	Before any real-world / production use
Low	Resident notifications (email/SMS/in-app)	Feature backlog
Low	Modern SPA frontend	Feature backlog, only if justified by user feedback